

4 3 2 1 5
6 7 8 9

4 3 2 1 5 6 7 8 9

X	X	X	X	X	X	X	X	X
---	---	---	---	---	---	---	---	---

1 4 5 6 7 8 9

1 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1. remove
2. ignore
3. visited
4. self loop
5. Add nans

1. remove
2. ignore
3. visited
4. self loop
5. Add nans

```
public void BFS() {  
    HashSet<Integer> visited = new HashSet<>();  
    Queue<Integer> q = new LinkedList<>();  
    for (int src : map.keySet()) {  
        q.add(src);  
        while (!q.isEmpty()) {  
            // 1. remove  
            int r = q.poll();  
            // 2. Ignore if Already visited  
            if (visited.contains(r)) {  
                continue;  
            }  
            // 3. marked visited  
            visited.add(r);  
            // 4. self work  
            System.out.print(r + " ");  
            // 5. add unvisited nbrs  
            for (int nbrs : map.get(r).keySet()) {  
                if (!visited.contains(nbrs)) {  
                    q.add(nbrs);  
                }  
            }  
        }  
    }  
    System.out.println();  
}
```

BFS

Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true

0	2k
1	3k
2	
3	
4	

1 2 3 4
0 1
0 2
0 3
1 4

2k=0
3k=2

0	1
0	2
0	3
1	4

2,0=0
2,1=3

3,0=1
3,1=4

for (int i = 0; i < edges.length; i++) {
 int a = edges[i][0];
 int b = edges[i][1];
 map.get(a).add(b);
 map.get(b).add(a);
}

There is an **undirected** graph with n nodes, where each node is numbered between 0 and $n - 1$. You are given a 2D array `graph`, where `graph[u]` is an array of nodes that node `u` is adjacent to. More formally, for each `v` in `graph[u]`, there is an undirected edge between node `u` and node `v`. The graph has the following properties:

- There are no self-edges (`graph[u]` does not contain `u`).
- There are no parallel edges (`graph[u]` does not contain duplicate values).
- If `v` is in `graph[u]`, then `u` is in `graph[v]` (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes `u` and `v` such that there is no path between them.

A graph is **bipartite** if the nodes can be partitioned into two independent sets `A` and `B` such that **every** edge in the graph connects a node in set `A` and a node in set `B`.

Return `true` if and only if it is **bipartite**.

2 colors
A, B

1 4 5 6 7 8 9

1 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

1

acyclic

cycle

odd

even

cycle
odd: odd/even

1 2 3 4 5 6 7 8 9

1 2 3 4 5 6 7 8 9

vtx	dis
1	0
2	1
3	1
4	2
5	2

- 1. remove
- 2. ignore
- 3. visited
- 4. self loop
- 5. Add nans

vtx	dis
1	0
2	1
3	2
4	1
5	2

```
public boolean isBipartite(int[][] graph) {  
    HashMap<Integer, Integer> visited = new HashMap<>();  
    Queue<BipartitePair> q = new LinkedList<>();  
    for (int src = 0; src < graph.length; src++) {  
        if (visited.containsKey(src)) {  
            continue;  
        }  
        q.add(new BipartitePair(src, 0));  
        while (!q.isEmpty()) {  
            // 1. remove  
            BipartitePair rp = q.poll();  
            // 2. Ignore if Already visited  
            if (visited.containsKey(rp.vtx)) {  
                continue;  
            }  
            // 3. marked visited  
            visited.put(rp.vtx, rp.dis);  
            // 4. self work  
            // 5. add unvisited nbrs  
            for (int nbrs : graph[rp.vtx]) {  
                if (!visited.containsKey(nbrs)) {  
                    q.add(new BipartitePair(nbrs, rp.dis + 1));  
                }  
            }  
        }  
    }  
    return true;  
}
```

v	a
0	0
1	1
2	1
3	1

Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]